

Исследование строковых сортировок

Введение

В этой работе я сравнивал обычные алгоритмы сортировки строк и специальные алгоритмы, которые лучше подходят именно для строк.

Для чисел стандартные сортировки работают хорошо, потому что сравнение двух чисел считается за постоянное время. Для строк ситуация другая: строки сравниваются посимвольно, и если у них длинный общий префикс, то даже одно сравнение может оказаться дорогим. Из-за этого обычная оценка вроде $O(n \log n)$ уже не даёт полной картины, потому что важно не только количество сравнений, но и то, сколько символов реально было просмотрено.

Поэтому в работе я решил сравнить не только обычные QUICKSORT и MERGESORT, но и специальные строковые алгоритмы:

- STRING QUICKSORT;
- STRING MERGESORT с использованием длины наибольшего общего префикса;
- MSD RADIX SORT;
- MSD RADIX SORT с переключением на STRING QUICKSORT для маленьких подмассивов.

Для каждого алгоритма измерялись:

- время выполнения;
- количество посимвольных сравнений.

Подготовка тестовых данных

Для генерации массивов строк был реализован класс `StringGenerator`. Он создаёт строки из алфавита, состоящего из:

- заглавных латинских букв;
- строчных латинских букв;
- цифр;
- специальных символов `!@#%: ; ^&*() - .`.

Длина строк выбиралась от 10 до 200 символов. Размеры массивов брались от 100 до 3000 с шагом 100.

Для экспериментов использовались следующие типы массивов:

- случайные;
- обратно отсортированные;
- почти отсортированные;
- массивы с длинными общими префиксами.

Случайные массивы создавались полностью случайно. Обратно отсортированные получались сортировкой случайного массива и последующим разворотом. Почти отсортированные массивы я получал так: сначала сортировал массив, а потом делал небольшое число случайных перестановок пар элементов. Для проверки поведения алгоритмов на похожих строках я отдельно генерировал массивы, в которых у строк специально задавался общий длинный префикс.

Организация замеров

Для измерений был реализован класс `StringSortTester`. Он запускал сортировки на одинаковых входных данных и сохранял результаты.

Для каждого запуска фиксировались:

- название алгоритма;
- тип данных;
- размер массива;
- число посимвольных сравнений;
- время работы в миллисекундах.

Результаты сохранялись в CSV-файл, а потом по ним строились графики.

Какие алгоритмы сравнивались

1. Обычный QuickSort

Это стандартная быстрая сортировка, в которой строки сравниваются как обычные строки. Такой алгоритм не хранит дополнительную информацию о совпадающих префиксах, поэтому одинаковые начала строк могут сравниваться много раз.

2. Обычный MergeSort

Обычная сортировка слиянием тоже использует стандартное строковое сравнение. Алгоритм стабилен и предсказуем по структуре работы, но он точно так же не умеет использовать уже известную информацию о совпавших символах.

3. STRING QUICKSORT

Эта версия работает не сразу со всей строкой, а с символом на текущей позиции. Массив делится на три части:

- строки с символом меньше опорного;
- строки с символом, равным опорному;
- строки с символом больше опорного.

Для средней части сортировка продолжается уже по следующему символу. За счёт этого алгоритм особенно хорошо работает на строках с длинными общими префиксами.

4. STRING MERGESORT

Эта версия сортировки слиянием использует длину наибольшего общего префикса. Идея в том, что если уже известно, сколько символов совпадает, то не нужно заново сравнивать эти символы в следующих шагах. Именно на этом алгоритме особенно интересно сравнивать теорию и практику.

5. MSD RADIX SORT

Этот алгоритм сортирует строки по символам, начиная с первого символа. Он разбивает строки по корзинам и потом рекурсивно сортирует каждую группу по следующему символу. По сути, здесь уже нет обычного сравнения строк как в quicksort или mergesort.

6. MSD RADIX SORT с переключением

У обычного MSD radix sort есть слабое место: на маленьких подмассивах его служебные действия уже могут стоить слишком дорого. Поэтому в улучшенной версии при маленьком размере подмассива выполняется переключение на STRING QUICKSORT. На практике такой гибрид обычно работает лучше чистого MSD radix sort.

Ожидания перед экспериментом

До запуска экспериментов ожидалось примерно следующее.

На случайных строках обычные алгоритмы должны показывать нормальные результаты, и разница со строковыми версиями может быть не очень большой.

На массивах с длинными общими префиксами ожидалось преимущество у STRING QUICKSORT, STRING MERGESORT и MSD RADIX SORT, потому что они лучше работают со структурой строк и не делают столько лишних посимвольных сравнений.

Отдельно ожидалось, что версия MSD RADIX SORT с переключением будет стабильнее, чем чистый MSD radix sort, потому что на маленьких подмассивах ей не придётся тратить время на лишнюю работу.

Результаты экспериментов

Ниже приведены графики времени работы и количества посимвольных сравнений.

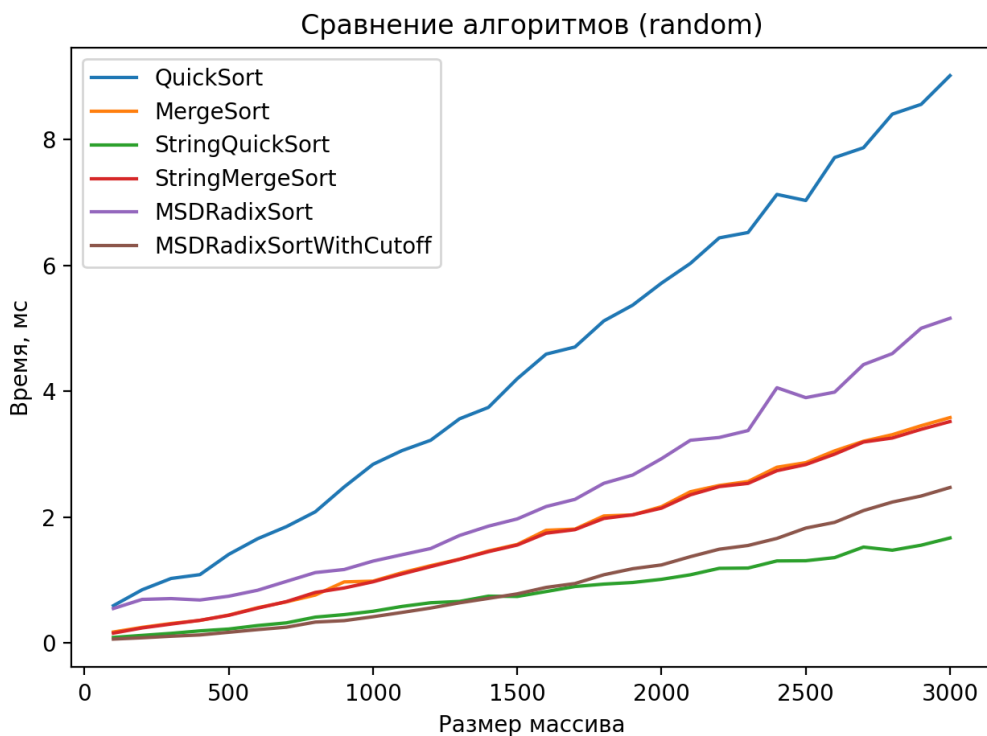


Рис. 1: Время работы на случайных данных

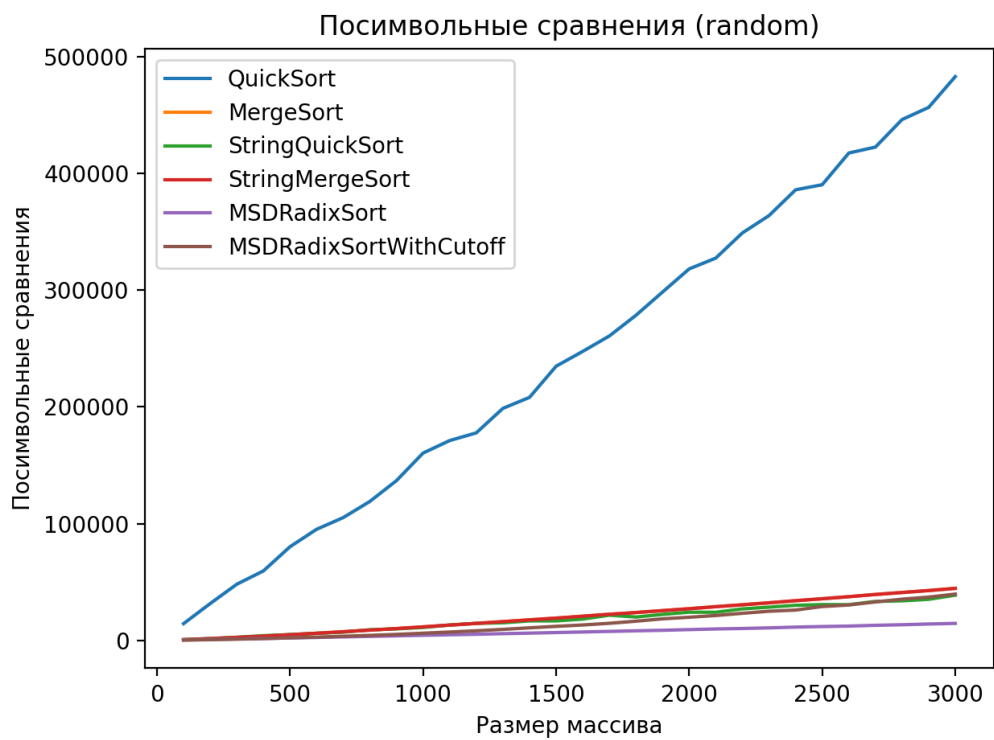


Рис. 2: Посимвольные сравнения на случайных данных

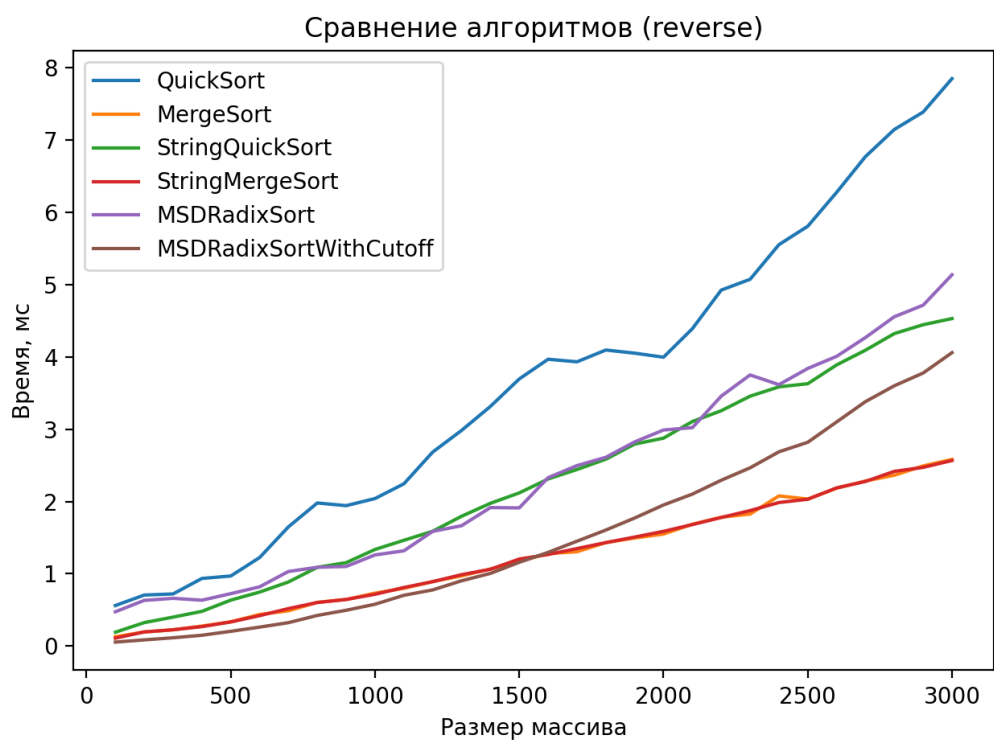


Рис. 3: Время работы на обратно отсортированных данных

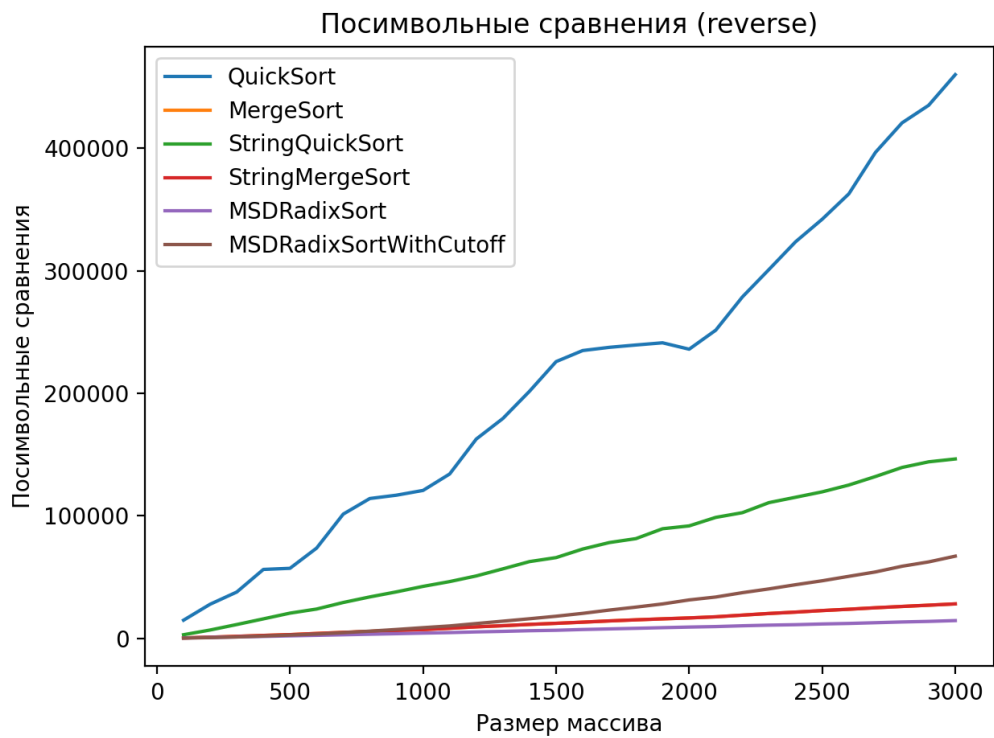


Рис. 4: Посимвольные сравнения на обратно отсортированных данных

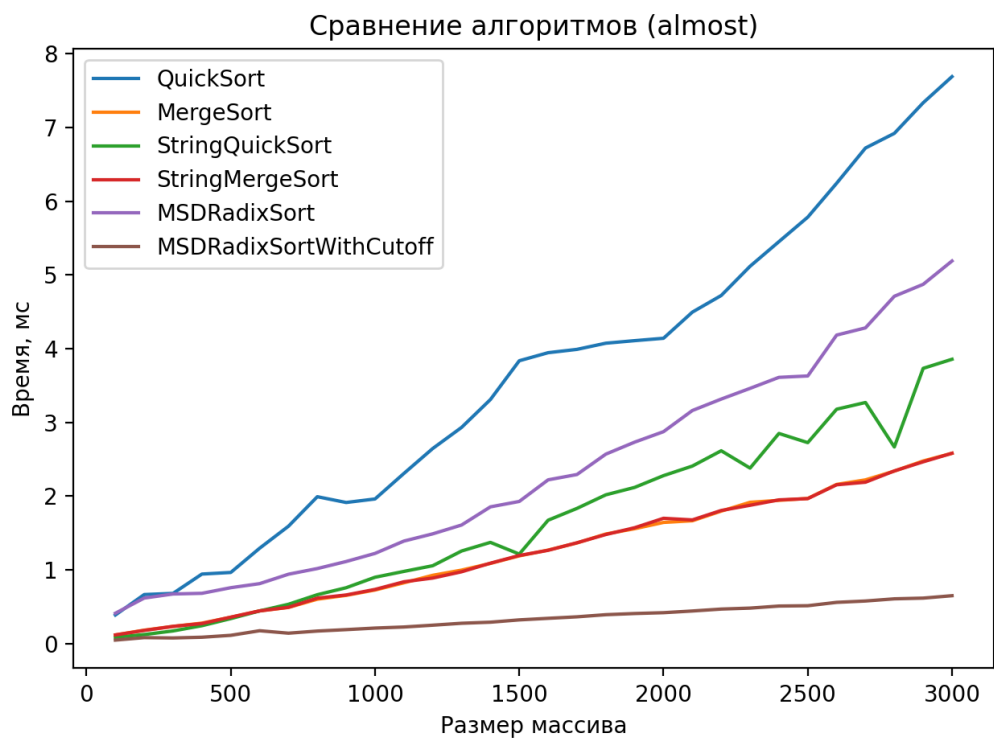


Рис. 5: Время работы на почти отсортированных данных

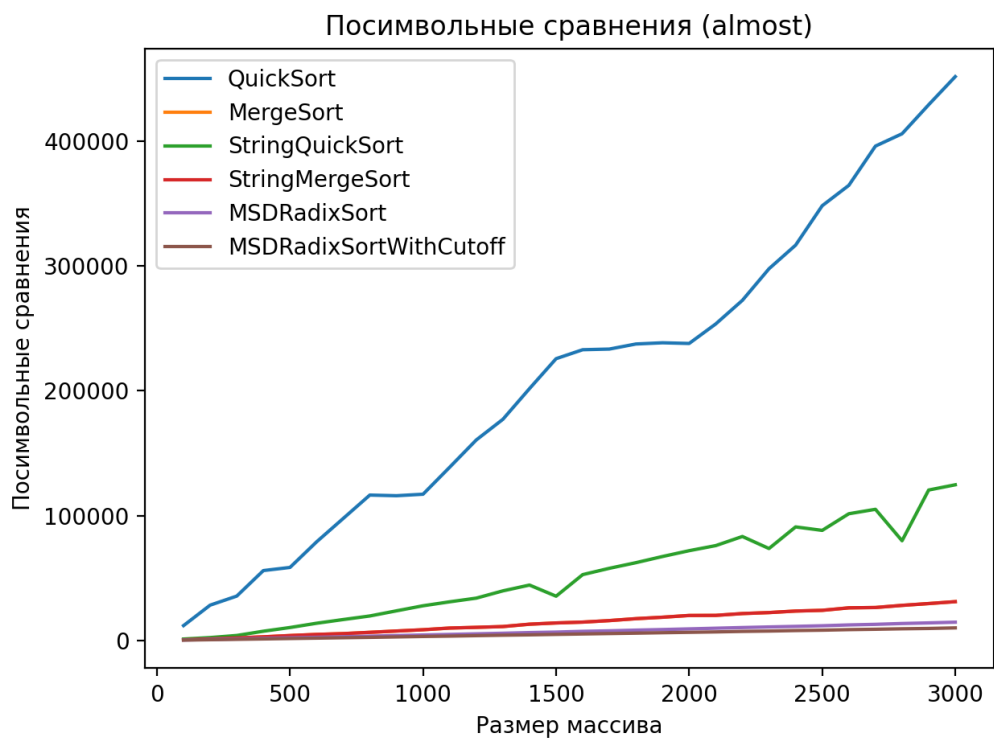


Рис. 6: Посимвольные сравнения на почти отсортированных данных

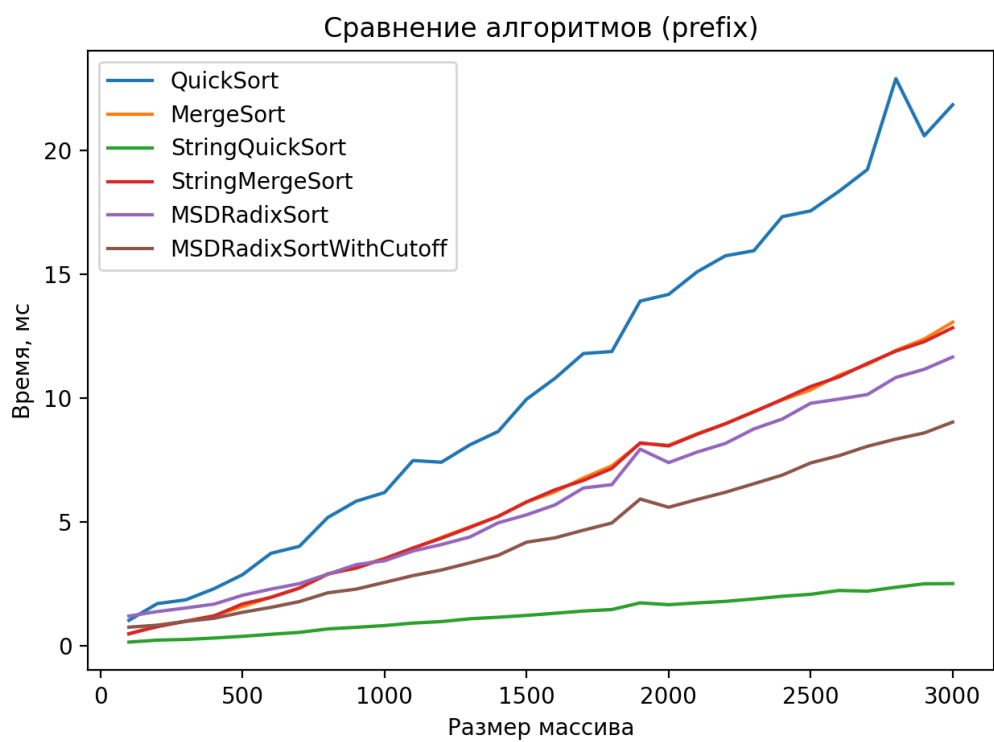


Рис. 7: Время работы на данных с общими префиксами

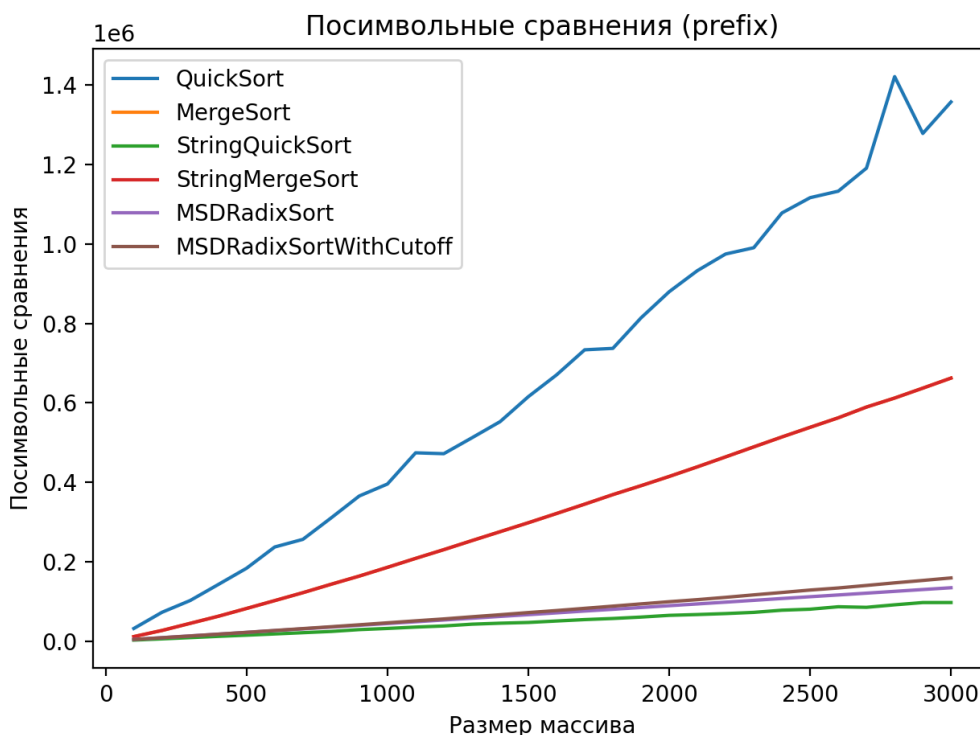


Рис. 8: Посимвольные сравнения на данных с общими префиксами

Анализ результатов

По случайным данным обычные сортировки вели себя более-менее ожидаемо. Они не выглядели катастрофически плохо, и на таких массивах разница между обычными и адаптированными алгоритмами не всегда очень большая.

На обратно отсортированных и почти отсортированных данных различия уже заметнее. Особенно интересно смотреть не только на время, но и на число посимвольных сравнений. Иногда по времени алгоритмы выглядят близко, но по числу сравнений уже хорошо видно, что один из них делает намного больше лишней работы.

Самая большая разница появилась на массивах с длинными общими префиксами. Именно здесь обычные QUICKSORT и MERGESORT начинают проигрывать, потому что снова и снова сравнивают одинаковые начала строк. Специальные строковые алгоритмы используют эту структуру заметно лучше.

STRING QUICKSORT показал себя хорошо на строках с длинными совпадающими началами. STRING MERGESORT тоже дал более приятные результаты по числу посимвольных сравнений по сравнению с обычным mergesort. MSD RADIX SORT оказался сильным вариантом на больших массивах, а версия с переключением на STRING QUICKSORT в маленьких подмассивах выглядела наиболее практичной.

Сравнение с теоретическими оценками

Полученные результаты в целом согласуются с теорией.

Для обычных сортировок оценка по количеству сравнений вроде $O(n \log n)$ ещё не означает, что сортировка будет хорошей именно на строках. Если каждое сравнение может проверять длинный общий префикс, реальная стоимость работы становится выше.

STRING MERGESORT как раз должен выигрывать на строках за счёт использования длины общего префикса, и это видно по экспериментам.

STRING QUICKSORT и MSD RADIX SORT тоже соответствуют ожиданиям: на данных, где строки похожи, они работают лучше, чем обычные алгоритмы.

Отдельно можно сказать, что MSD RADIX SORT с переключением выглядит наиболее удачной практической версией, потому что сочетает преимущества поразрядной сортировки и string quicksort.

Вывод

По итогам работы можно сделать довольно простой вывод: для строк обычные алгоритмы сортировки не всегда являются лучшим выбором, особенно если строки имеют длинные общие префиксы.

Если данные похожи друг на друга, специализированные строковые алгоритмы начинают заметно выигрывать как по времени, так и по числу посимвольных сравнений.

Из всех рассмотренных вариантов наиболее удачными по совокупности результатов оказались:

- STRING QUICKSORT;
- STRING MERGESORT;
- MSD RADIX SORT с переключением на STRING QUICKSORT.

То есть общий итог такой: для строк действительно имеет смысл использовать специальные методы сортировки, а не ограничиваться обычными quicksort и mergesort.

Что приложено

К работе приложены:

- реализация класса `StringGenerator`;
- реализация класса `StringSortTester`;
- реализации всех исследованных алгоритмов сортировки;
- файл с результатами замеров;
- графики;
- ID посылок;
- ссылка на публичный репозиторий.

ID посылки

- A1m: 375975710
- A1q: 375975966
- A1r: 375977019
- A1rq: 375977139

Ссылка на публичный репозиторий

<https://github.com/thebestwillbuyrr/string-sorting-analysis>